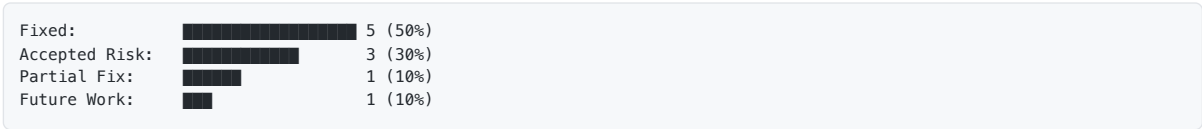


Risk Register — MediConnect

Risk Register

ID	Risk	Component	Impact	Likelihood	Severity	HIPAA Control	Mitigation	Residual Risk	Status
AS-01	Unauthorized access to patient chat history	chatController.js	Critical	High	Critical	§164.312(a)(1)	Room ownership check	None	Fixed
AS-03	Unauthorized prescription termination	prescriptionController.js	Critical	High	Critical	§164.312(c)(1)	Doctor ownership check	None	Fixed
AS-04	Unread message metadata leakage	chatController.js	High	High	Critical	§164.312(a)(1)	Use JWT userId	None	Fixed
AS-06	Admin privilege escalation via registration	authController.js	Critical	High	Critical	§164.308(a)(4)	Role whitelist	None	Fixed
AS-05	Missing role authorization on Rx routes	prescriptionRoutes.js	High	Medium	High	§164.308(a)(4)	authorizeRoles middleware	None	Fixed
AS-07	JWT valid after logout	authController.js	Medium	Medium	Medium	§164.312(a)(2)(iii)	HttpOnly + SameSite cookies	Token valid for 24h	Accepted
AS-08	No rate limiting on login	authController.js	Medium	High	Medium	§164.308(a)(5)(ii)(C)	Brute-force detection alert	Blocking not implemented	Partial
SG-01	Missing CSRF protection	app.js	Medium	Low	Medium	—	SameSite=Lax cookies	CSRF possible on older browsers	Accepted
SG-02	TLS bypass in DB config	db/index.js	Low	Low	Low	§164.312(e)(1)	Dev environment only	Production must enable strict TLS	Accepted
DEP-01	Vulnerable npm dependencies	package.json	Variable	Medium	Low-High	—	Monitoring via npm audit	Known CVEs in transitive deps	Future

Risk Distribution



Accepted Risk Justifications

AS-07 — Token Revocation

- **Why Accepted:** Implementing a token blacklist requires Redis or database-backed session store, which adds infrastructure complexity. The current 24-hour expiry + HttpOnly + SameSite cookies provide reasonable protection for a prototype.
- **Residual Risk:** A stolen token remains valid for up to 24 hours after logout.
- **Future Mitigation:** Implement Redis-backed token blacklist.

SG-01 — CSRF Protection

- **Why Accepted:** The application uses SameSite=Lax cookies, which prevent CSRF on cross-origin POST requests in modern browsers. The API is consumed by a same-origin React frontend.
- **Residual Risk:** Older browsers (pre-2020) may not enforce SameSite.
- **Future Mitigation:** Add double-submit cookie CSRF pattern.

SG-02 — TLS Bypass

- **Why Accepted:** Development environment uses local PostgreSQL without TLS. This setting does not affect production.
- **Residual Risk:** None in development. Must be configured for production.
- **Future Mitigation:** Environment-specific SSL configuration.